B.M.S. COLLEGE OF ENGINEERING

(Autonomous College Affiliated to Visvesvaraya Technological University, Belgaum)
Bull Temple Road, Basavanagudi, Bengaluru - 560019

Department of Electronics & Communication Engineering (ECE)

A Project Report on

Synchronous Digital Clock Design and Verification

Submitted for the partial fulfilment of the requirement of Alternate Assessment for the course

System Verilog and Verification [23EC6PESV]

Submitted by

SHREEYA SACHIN KIRDAT 1BM23EC249

Under the support and Guidance of

Prof. Ajaykumar Devarapalli
Assistant Professor, Dept. of EIE
BMSCE

February 2026

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Department of Electronics & Communication Engineering (ECE)

DECLARATION

I, **Shreeya Sachin Kirdat (1BM23EC249)**, hereby declare that the report titled *"Synchronous Digital Clock Design and Verification"* is my genuine and original work carried out as part of the activity plan for the course **System Verilog and Verification (23EC6PE2SV)** in the Department of Electronics and Communication Engineering (ECE), BMS College of Engineering, during the academic year 2025–26.

I further declare that this work has not been submitted elsewhere for the award of any degree or diploma. All the corrections and suggestions indicated during the Internal Assessments have been duly incorporated into this report.

Shreeya Sachin Kirdat

1BM23EC249

Table of Contents

1. Problem Statement 4
2. GitHub Repo Link 4
3. RTL Design 5
4. Verification 6
5. Coverage 9
6. Waveform 10
7. Code Snippets 12
8. Conclusion 13
   8.1 Key Learnings 13

1. Problem Statement

Design and verify a Digital Clock module using SystemVerilog. The clock consists of two synchronized 6-bit counters representing seconds and minutes, each counting from 0 to 59.

Functional Requirements:

- Seconds Counter: A 6-bit counter that increments by 1 on every positive edge of the clock signal. Valid range: 0 to 59.
- Minutes Counter: A 6-bit counter that increments by 1 only when the seconds counter rolls over from 59 to 0. Valid range: 0 to 59.
- Rollover Behavior:
  - When seconds reach 59, the next clock cycle resets seconds to 0 and increments minutes.
  - When both seconds and minutes are 59, the next clock cycle resets both counters to 0 (simulating hour transition).
- Synchronous Reset: An active-high reset signal that initializes both counters to 0 on the next clock edge.

Design Specifications:

- Clock frequency: Assume 1 Hz for real-time simulation (adjustable for faster verification).
- Both counters must be synchronous to the same clock domain.
- No asynchronous signals except for potential power-on reset.

2. GitHub Repo Link

URL: https://github.com/Shreeya-1030/AAT2-Digital-Clock

3. RTL Design

```
`timescale 1ns/1ps


module digital_clock (
input  logic       clk,     input
logic       reset,     output
logic [5:0]  seconds,
   output logic [5:0]  minutes
);

   always_ff @(posedge clk) begin
if (reset) begin            seconds <=
6'd0;            minutes <= 6'd0;
       end        else begin            if
(seconds == 6'd59) begin
              seconds <= 6'd0;

              if (minutes == 6'd59)
minutes <= 6'd0;
              else                    minutes
<= minutes + 1;
          end            else begin
seconds <= seconds + 1;
          end
end
   end

endmodule
```

4. Verification

Interface:

```
`timescale 1ns/1ps

interface clock_if (input logic clk);

   logic reset;     logic
[5:0] seconds;     logic
[5:0] minutes;

   modport DUT (
      input  clk, reset,
      output seconds, minutes
   );

   modport TB (
      input  clk,
output reset,
      input  seconds, minutes
   );

endinterface
```

Testbench:

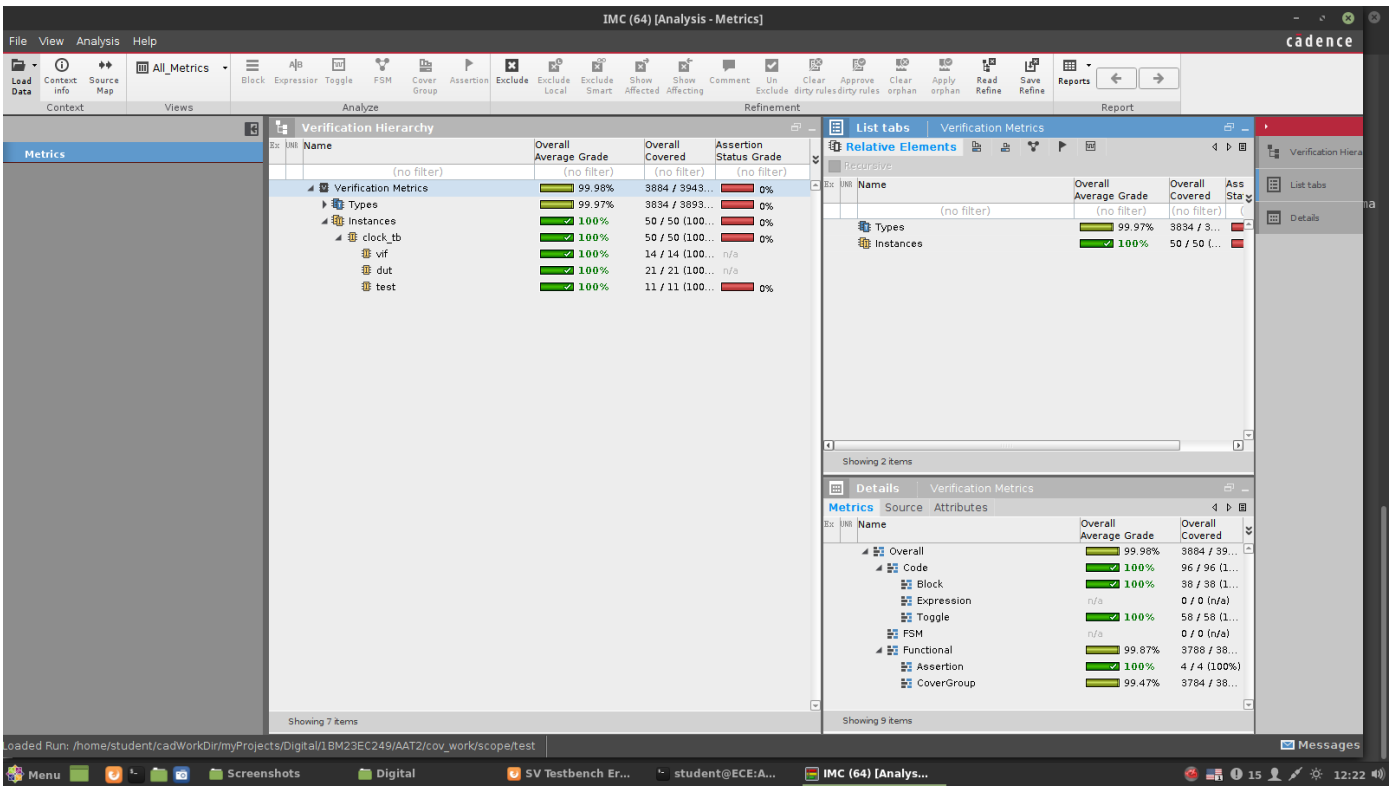
```
`timescale 1ns/1ps

program clock_test (clock_if.TB vif);

   covergroup cg @(posedge vif.clk);

      cp_seconds : coverpoint vif.seconds {
bins sec_vals[] = {[0:59]};
         bins sec_rollover = (59 => 0);
      }

      cp_minutes : coverpoint vif.minutes {
bins min_vals[] = {[0:59]};
```

```
            bins min_rollover = (59 => 0);
        }

        cross cp_seconds, cp_minutes;

    endgroup

    cg coverage_inst = new();

    property sec_limit;
        @(posedge vif.clk) vif.seconds <= 6'd59;
endproperty

    property min_limit;
        @(posedge vif.clk) vif.minutes <= 6'd59;
endproperty

    assert property (sec_limit)         else
$error("Seconds exceeded 59!");

    assert property (min_limit)
        else $error("Minutes exceeded 59!");

    initial begin         vif.reset = 1;
repeat (2) @(posedge vif.clk);
        vif.reset = 0;

        repeat (3700) @(posedge vif.clk);

        vif.reset = 1;         repeat (2)
@(posedge vif.clk);
        vif.reset = 0;

        repeat (200) @(posedge vif.clk);

        $display("Simulation Completed Successfully.");
        $finish;
    end

endprogram
```

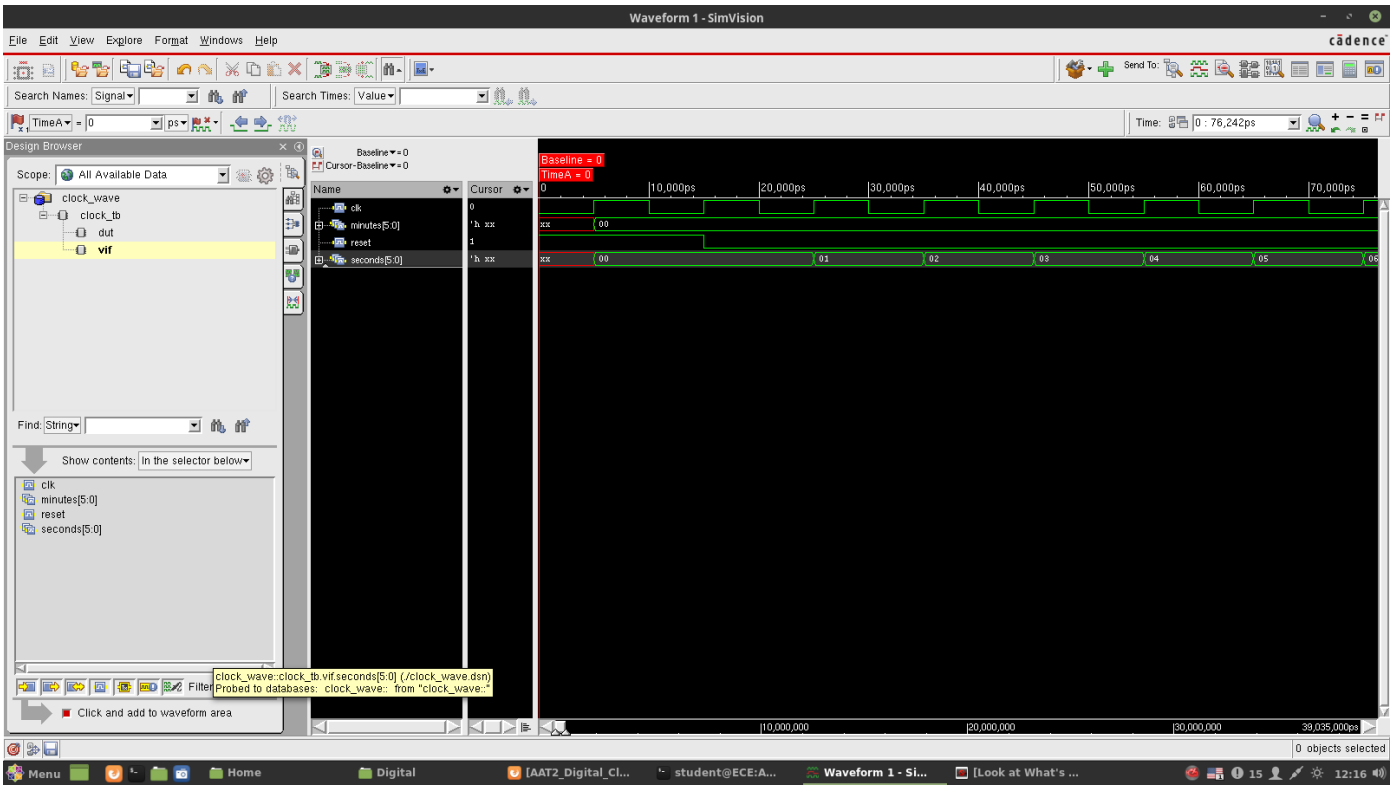
Top Module:

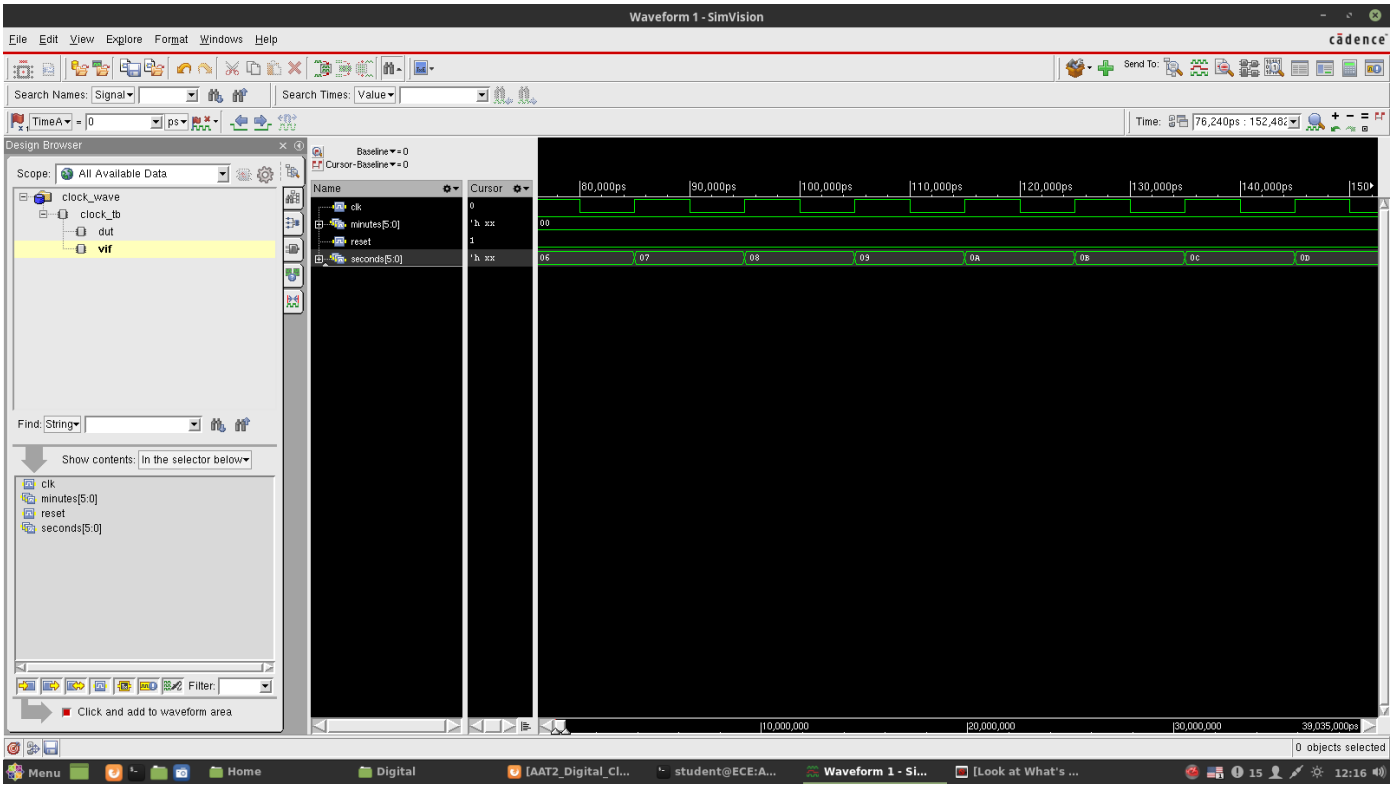
```
`timescale 1ns/1ps

module clock_tb;

    logic clk; // Declare the clock signal here

    // Pass the clock into the interface
clock_if vif(clk);

    digital_clock dut (
        .clk     (vif.clk),
        .reset   (vif.reset),
        .seconds (vif.seconds),
        .minutes (vif.minutes)
    );

    clock_test test (vif);

    // Clock generation (drive the local clk variable, not vif.clk)
initial begin         clk = 0;         forever #5 clk = ~clk;     end

    // VCD Dump
    initial begin
        $dumpfile("clock_wave.vcd");
$dumpvars(0, clock_tb);
    end

endmodule
```

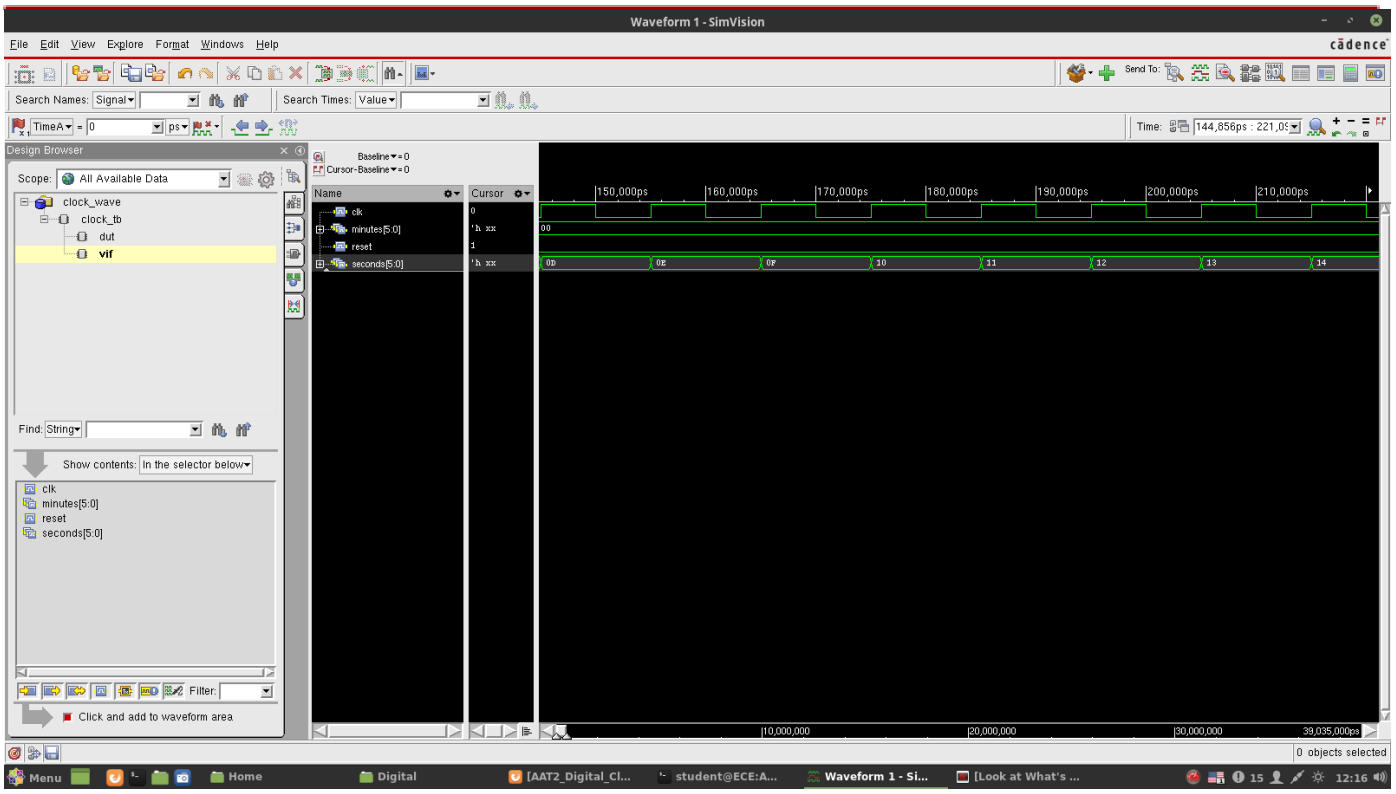
5. Coverage


IMC (64) [Analysis - Metrics]
File View Analysis Help
cadence
Verification Hierarchy
Verification Metrics 99.98% 3884 / 3943... 0%
Types 99.97% 3834 / 3893... 0%
Instances 100% 50 / 50 (100... 0%
clock_tb 100% 50 / 50 (100... 0%
vif 100% 14 / 14 (100... n/a
dut 100% 21 / 21 (100... n/a
test 100% 11 / 11 (100... 0%
Showing 7 items
List tabs Verification Metrics
Relative Elements
Types 99.97% 3834 / 3...
Instances 100% 50 / 50 (...
Showing 2 items
Details Verification Metrics
Metrics Source Attributes
Overall 99.98% 3884 / 39...
Code 100% 96 / 96 (1...
Block 100% 38 / 38 (1...
Expression n/a 0 / 0 (n/a)
Toggle 100% 58 / 58 (1...
FSM n/a 0 / 0 (n/a)
Functional 99.87% 3788 / 38...
Assertion 100% 4 / 4 (100%)
CoverGroup 99.47% 3784 / 38...
Showing 9 items
Loaded Run: /home/student/cadWorkDir/myProjects/Digital/1BM23EC249/AAT2/cov_work/scope/test
Messages

6. Waveform


Waveform 1 - SimVision
File Edit View Explore Format Windows Help
cadence
Design Browser
Scope: All Available Data
clock_wave
clock_tb
dut
vif
Find: String
Show contents: In the selector below
clk
minutes[5:0]
reset
seconds[5:0]
Click and add to waveform area
Baseline = 0
TimeA = 0
Name
Cursor
clock_wave::clock_tb.vif.seconds[5:0] (/clock_wave.dsn)
Probed to databases: clock_wave:: from "clock_wave::"
0 objects selected


Waveform 1 - SimVision
File Edit View Explore Format Windows Help
cadence
Design Browser
Scope: All Available Data
clock_wave
clock_tb
dut
vif
Find: String
Show contents: In the selector below
clk
minutes[5:0]
reset
seconds[5:0]
Click and add to waveform area
Name
Cursor
0 objects selected


Waveform 1 - SimVision
File Edit View Explore Format Windows Help
cadence
Search Names: Signal
Search Times: Value
Design Browser
Scope: All Available Data
clock_wave
clock_tb
dut
vif
Find: String
Show contents: In the selector below
clk
minutes[5:0]
reset
seconds[5:0]
Filter:
Click and add to waveform area
Name
Cursor
150,000ps
160,000ps
170,000ps
180,000ps
190,000ps
200,000ps
210,000ps
10,000,000
20,000,000
30,000,000
39,035,000ps
0 objects selected
Menu
Home
Digital
[AAT2_Digital_Cl...
student@ECE:A...
Waveform 1 - Si...
[Look at What's ...
12:16

7. Code Snippets

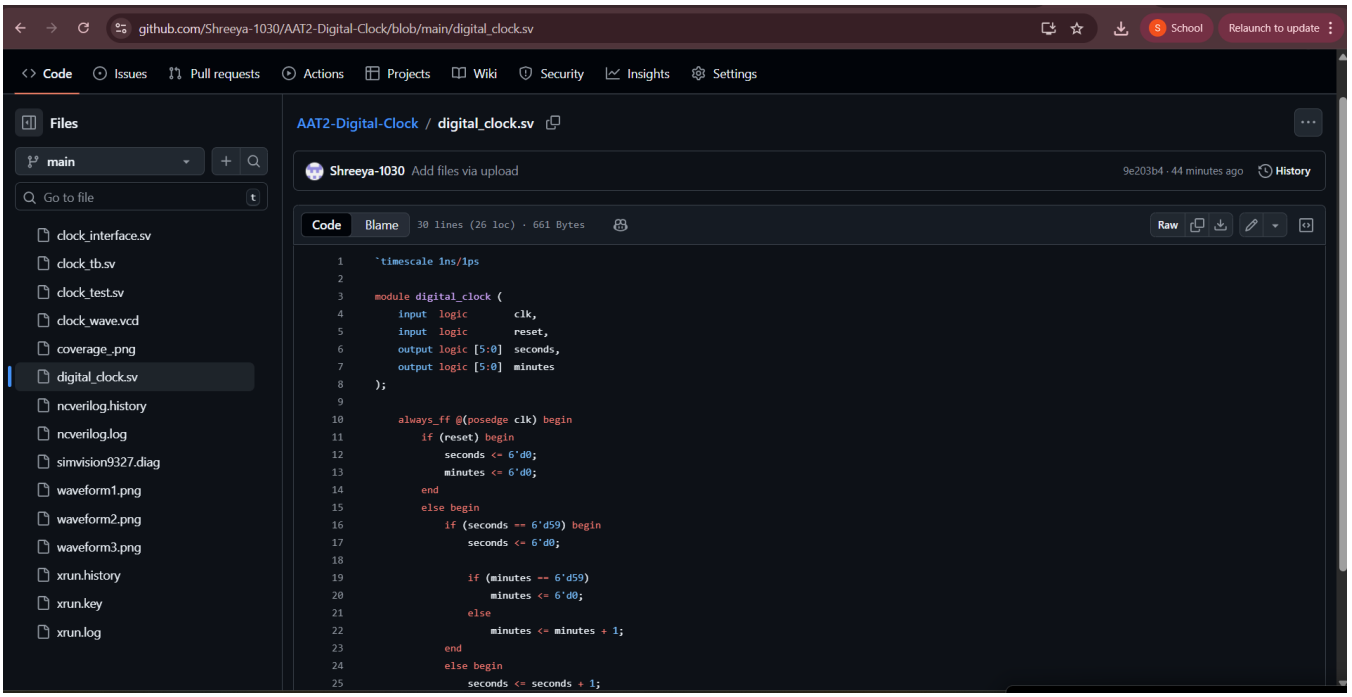


Figure 8.1 – GitHub Repo

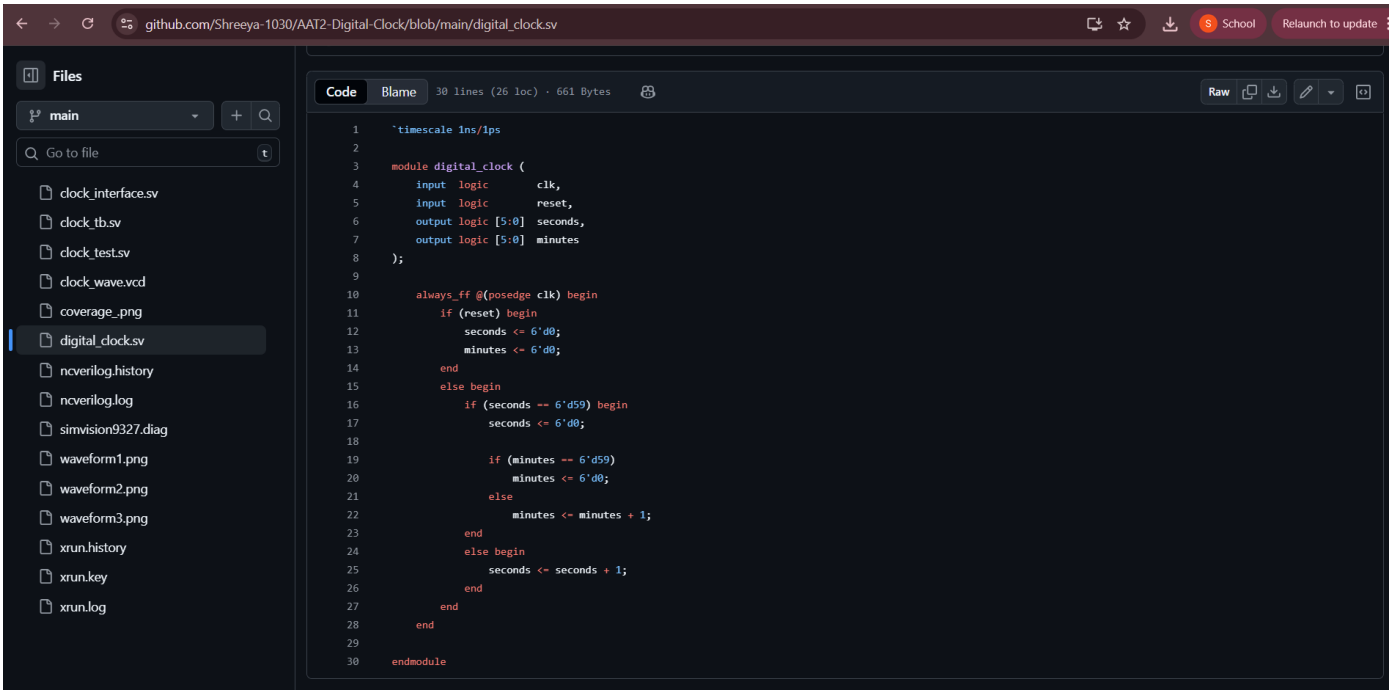


Figure 8.2 – digital_clock.sv

8. Conclusion

The Synchronous Digital Clock was successfully designed and verified using SystemVerilog. The module correctly implements two synchronized 6-bit counters for seconds and minutes with proper rollover behavior (59 → 0). The minutes counter increments only when the seconds counter resets, and both counters reset correctly upon reaching 59. A synchronous active-high reset ensures predictable initialization.

The verification environment was built using interfaces, program blocks, assertions, and functional coverage. Simulation waveforms confirm correct rollover behavior, and coverage results demonstrate complete functional verification of the design.

8.1 Key Learnings

- **Synchronous Design Principles:**
  Developed a clear understanding of designing clock-driven sequential logic using always_ff and non-blocking assignments.

- **Counter Dependency Logic:**
  Learned how to implement multi-level rollover logic where one counter's transition controls another counter.

- **Structured Verification Methodology:**
  Gained experience in building a modular verification environment using interfaces and program blocks to avoid race conditions.

- **Functional Coverage and Assertions:**
  Understood the importance of value, transition, and cross coverage along with assertionbased checks to ensure complete validation.

- **Simulation and Debug Analysis:**
  Improved ability to interpret waveform outputs and coverage reports to confirm correctness and detect anomalies.